DarkDrop V4 — Security Audit #4: Post-Revoke Review

Program ID: `GSig1QYVwPVhHF6oVEwhadAwdWjTqtq6H5cSMEkfAgkU` **Audit date:** April 20, 2026 **Scope:** V3 Note Pool layer + `revoke_drop` instruction + `DepositReceipt` PDA + `create_drop` remaining_accounts extension + counter interaction invariants across all withdrawal paths + re-verification of Audit #3 open findings + privacy analysis of revoke path. **Prior audits:** #1 (April 6, 2026), #2 (April 7, 2026), #3 (April 8, 2026) **Framework:** Anchor 0.30.1, groth16-solana 0.0.3 (pinned, unchanged since Audit #2), light-hasher 4.0.0 **Binary under review:** 601,576 bytes, commit after April 20 revoke deploy

---

Severity Scale

Level	Definition
CRITICAL	Funds can be drained or stolen. Immediate exploit path exists.
HIGH	Significant financial loss possible under realistic conditions.
MEDIUM	Unexpected behavior or limited financial impact. Exploitable under specific conditions.
LOW	Best practice violation. No direct exploit but increases attack surface.
INFORMATIONAL	Code quality, gas optimization, or design notes. No security impact.

Executive Summary

This audit covers two feature expansions shipped since Audit #3:

1. **V3 Note Pool layer** — recursive privacy via a second-layer Merkle mixer (`initialize_note_pool` , `deposit_to_note_pool` , `claim_from_note_pool` , plus `NotePool` / `NotePoolTree` / `PoolNullifierAccount` state and the V3 circuit with 4 public inputs).
2. **`revoke_drop` instruction + `DepositReceipt` PDA** — depositor fallback for unclaimed drops, deployed April 20, 2026. Adds a `remaining_accounts` extension to `create_drop` for backward-compatible receipt creation, a new `poseidon_hash_1` helper, a `short-revoke-timeout` Cargo feature for localnet testing, and five new error codes (6014–6018).

The core protocol security properties from Audit #3 are preserved in both expansions. ZK verification, nullifier double-spend prevention, Poseidon binding, treasury obligation accounting, and direct-lamport privacy all continue to hold across the expanded surface.

Findings: 0 CRITICAL, 0 HIGH, 1 MEDIUM (**fixed during this cycle**), 4 LOW, 4 INFORMATIONAL.

The MEDIUM finding — **M-01 orphan `DepositReceipt` after normal claim** — is a **rent-lock bug** that was identified, analyzed, and **fixed within this audit cycle** via a new `close_receipt` instruction. All tests pass on localnet and the fix has been deployed to devnet. An initial version of M-01 in this document overstated the privacy impact by claiming a full deposit→recipient chain; that claim has been corrected — V2 continues to cryptographically hide the leaf↔nullifier mapping, so the orphaned receipt leaks only deposit-side data, not the full claim chain. The deposit-side leak is now tracked separately as **L-04** (documentation-only, inherent to the receipt design).

Four LOW findings are open: three carryover from prior audits (drop_cap validation, authority rotation, main-tree root history zero-init) plus one new finding extending the root-history issue to the note pool tree. L-04 (deposit-side linkage inherent to receipts) is classified separately because it is a design property, not a code bug.

The `short-revoke-timeout` Cargo feature is **correctly implemented**: the production binary (`cargo build-sbf` , no feature flag) contains the 30-day constant `2_592_000` (verified by byte search for `0x278D00`) and does **not** contain the 5-second test constant. Feature flag leakage is not a risk.

Status updates for Audit #3 findings are in the next section.

Status Update: Audit #3 Findings

Audit #3 reported four new findings; all have been fixed and remain fixed in this audit's codebase.

ID (Audit 03)	Severity	Title	Status in Audit 04
H-01-NEW	HIGH	`admin_sweep` rug-pull with outstanding credit notes	**FIXED** — verified: `admin_sweep.rs:21-32` computes `outstanding = total_deposited - total_withdrawn` and caps sweep at `treasury_balance - outstanding - rent_exempt_min`. Cross-checked across `claim.rs`, `withdraw_credit.rs`, `revoke_drop.rs` — all three paths correctly increment `total_withdrawn`.
M-01-NEW	MEDIUM	CreditNote PDA leaks commitment on-chain	**FIXED** — verified: `claim_credit.rs:64-71` stores `Poseidon(amount_commitment, salt)` in the CreditNote. The salt is a user-provided 32-byte value and is not emitted in the `CreditCreated` event.
M-02-NEW	MEDIUM	`admin_sweep` emits no event	**FIXED** — `TreasurySweep` event with `authority`, `amount`, `timestamp` is emitted at `admin_sweep.rs:40-44`.
M-03-NEW	MEDIUM	`DropCreated` event leaks `amount_commitment` + `password_hash`	**FIXED** — verified: `create_drop.rs:129-135` emits only `leaf_index`, `leaf`, `merkle_root`, `timestamp`. The underscores on the `_amount_commitment` and `_password_hash` handler parameters confirm they are deliberately unused in the event.

Open LOW findings from Audit #2/#3 remain open:

ID	Severity	Title	Note
L-01-NEW (Audit 03)	LOW	No `drop_cap` validation	Re-verified: `initialize.rs` still accepts any u64 including 0 and `u64::MAX`.
L-02-NEW (Audit 03)	LOW	No authority rotation	Re-verified: `Vault::authority` is immutable.
L-03-NEW (Audit 03)	LOW	Root history zero-initialized (main tree)	Re-verified: `initialize.rs` sets only `root_history[0]`. Slots 1–29 remain `[0; 32]`.

`audits/README.md` fix tracker is stale and should be updated: it lists H-01-NEW, M-01-NEW, M-02-NEW, M-03-NEW as "Open". They are all FIXED per Audit 03 and this audit. The tracker update is part of Audit 04's deliverables.

New Findings (Audit 04)

[M-01] Orphan `DepositReceipt` after normal claim — rent lock (FIXED in this audit cycle)

Severity: MEDIUM **Files:**

- `instructions/create_drop.rs:66-128` (receipt creation branch)

- `instructions/revoke_drop.rs:30-65` (receipt close path, unreachable when nullifier exists)
- `instructions/close_receipt.rs` (**new instruction; fix for this finding**)

Status: ✅ FIXED — `close_receipt` instruction added during Audit 04 cycle. See "Fix" subsection below.

Description:

When a user calls `create_drop` with the new `remaining_accounts` extension, a `DepositReceipt` PDA is allocated at `[b"receipt", leaf]` storing `{bump, depositor, amount, created_at, leaf}`. The receipt is intended to enable `revoke_drop` if the claim code is lost.

If the drop is subsequently claimed normally (`claim_credit` → `withdraw_credit` or legacy `claim`), the receipt PDA is not automatically closed. `revoke_drop` cannot close it because the nullifier PDA at `[b"nullifier", nullifier_hash]` now exists (claim created it), and Anchor's `init` constraint on `revoke_drop.nullifier_account` fails before the `close = depositor` constraint on `deposit_receipt` is reached.

Consequence: the ~0.00151 SOL rent stored in the receipt PDA is permanently locked (no instruction can close it). At ~15–30 drops/day on devnet (per `ARCHITECTURE.md`), a non-trivial fraction of mainnet users opting into receipts results in a steady, ecosystem-level rent drip.

Privacy clarification (self-correction of initial write-up).

The initial version of this finding asserted that the orphaned receipt enabled the chain `depositor → leaf → claim_credit TX (by nullifier) → CreditNote.recipient`, claiming a full deposit-to-recipient linkage. **That chain is incorrect.** The step "leaf → claim_credit TX" requires mapping a leaf to its nullifier_hash, which V2 specifically prevents: the circuit proves knowledge of *some* leaf via the Merkle proof without revealing which one, and the nullifier_hash is the only on-chain signal that could be correlated back to a leaf — but doing so requires knowledge of the leaf preimage, which is private.

The correct privacy statement for the orphaned receipt is:

- `receipt.depositor` reveals the depositor's wallet.
- `receipt.leaf` reveals the leaf and its amount.
- These create a durable on-chain `depositor ↔ (leaf, amount)` link. This is **deposit-side** information only.
- The link does NOT compose with any future `claim_credit` TX's `nullifier_hash` or `CreditNote.recipient`, because V2 cryptographically hides which leaf a given claim corresponds to.

This deposit-side linkage is an **inherent property of creating a DepositReceipt** — it exists from the moment of deposit and persists in indexer history regardless of whether the receipt is later closed. It is not a bug; it is the privacy cost of opting into the revoke path. It is documented in ARCHITECTURE.md's "Privacy cost of revoking" section and classified as a separate LOW finding below ([L-04]).

The true fix-worthy component of M-01 is the **rent lock**.

Impact (revised):

- **Rent:** Per-drop ~0.00151 SOL permanently locked when a receipt-bearing drop is claimed (not revoked).
- **Privacy:** Deposit-side `depositor ↔ (leaf, amount)` leak, inherent to the receipt design, independent of whether the drop is later claimed or revoked. See [L-04] (new, below).

Attack scenario tested: Not an attack — structural rent lock. No active adversary involvement.

Fix (implemented during Audit 04 cycle):

A new `close_receipt` instruction was added:

```
Instruction: close_receipt
Args:     leaf ([u8; 32])
Accounts: deposit_receipt (mut, seeds = [b"receipt", leaf], close = depositor)
          depositor (signer, mut)
```

Handler (`instructions/close_receipt.rs`) performs one explicit authorization check:

```
require_keys_eq!(depositor.key(), receipt.depositor, InvalidDepositReceipt);
```

This check is essential — Anchor's `close = depositor` only specifies where lamports go; it does NOT verify that the signer matches the receipt's stored depositor. Without the explicit check, any signer could pass in any victim's receipt and steal the rent.

The close is **unconditional**: it does not verify whether the drop was claimed. Verifying via preimage would leak the preimage (same privacy cost as revoke, defeating the purpose of a low-cost close). Verifying via a nullifier_hash argument would put that value in the close_receipt TX, creating a new deposit↔close linkage observer. Unconditional close lets the depositor decide off-chain when it is safe to close; closing prematurely surrenders the revoke option, which is a UX footgun but not a security concern because only the depositor signs.

Rejected alternatives (explored during fix design):

- **Re-seed receipt by `nullifier_hash`.** Would allow `claim_credit` to auto-close via `remaining_accounts` since claim knows nullifier_hash. REJECTED: creates a systemic deposit↔claim linkage. Any observer who sees a claim_credit derives `PDA([b"receipt", nullifier_hash], program_id)` and looks up `(depositor, leaf, amount)` directly. This would be strictly worse than the original M-01 (on-chain trivially-computable linkage vs. requires indexer to find the receipt).
- **Dual PDA at `[b"receipt_nullifier", nullifier_hash]` pointing to leaf.** Same fundamental problem as above — any PDA keyed by nullifier_hash exposes the linkage at claim time.
- **Auto-close receipt inside `claim_credit`.** Would require `claim_credit` to reference the receipt PDA address in its account list. The address is a function of the leaf, so an observer matches the claim's account list against DropCreated events and learns which leaf was claimed — V2 privacy is broken. Incompatible with V2 regardless of receipt seed scheme.

The unconditional depositor-signed close_receipt is the only option that preserves V2 privacy. Its limitation is that it fixes only rent, not the inherent deposit-side privacy leak. This limitation is intrinsic to the receipt mechanism and is documented in [L-04] below.

Verification of the fix:

- `scripts/close-receipt-test.js` — E2E: create drop with receipt, claim normally, close_receipt, verify receipt closed, depositor rent recovered, nullifier PDA untouched, CreditNote PDA untouched.
- `scripts/security-revoke-tests.js` Tests G–K (new in this audit cycle):
  - **G** close_receipt by non-depositor → InvalidDepositReceipt (6017)
  - **H** close_receipt on nonexistent receipt → AccountNotInitialized
  - **I** close_receipt with mismatched leaf arg → ConstraintSeeds (2006)
  - **J** close_receipt then revoke → revoke fails (AccountNotInitialized on receipt)
  - **K** Double-close → second call fails (AccountNotInitialized)

[L-01] `note_pool_tree` root history zero-initialized

Severity: LOW **File:** `instructions/initialize_note_pool.rs:12-22`

Description:

Identical structural issue to Audit 03's L-03-NEW (which applied to the main Merkle tree) now applied to the note pool tree. `initialize_note_pool` initializes `filled_subtrees[]` correctly with `ZERO_HASHES[i]` but sets only `root_history[0] = ZERO_HASHES[MERKLE_DEPTH]`. Slots 1–29 of `root_history` remain `[0; 32]`.

The `is_known_root` function at `state.rs:239-250` iterates all 30 slots on every `claim_from_note_pool`, wasting compute units on zero-byte comparisons. Not exploitable — producing a valid Groth16 V3 proof against root `[0; 32]` would require breaking Poseidon preimage resistance.

Recommendation:

```
for i in 0..ROOT_HISTORY_SIZE {
    tree.root_history[i] = ZERO_HASHES[MERKLE_DEPTH];
}
```

Pair this fix with the existing L-03-NEW fix for the main tree; both are currently open.

[L-02] Carryover: `drop_cap` validation (Audit 03 L-01-NEW)

Severity: LOW **File:** `instructions/initialize.rs`

Re-verified still open. No `require!(drop_cap > 0 && drop_cap <= MAX)` check in `initialize_vault`. `drop_cap = 0` bricks the protocol; `drop_cap = u64::MAX` disables the safety cap.

[L-03] Carryover: authority rotation missing (Audit 03 L-02-NEW)

Severity: LOW **File:** `state.rs:60-77` (`Vault::authority` field)

Re-verified still open. No `update_authority` instruction. Compromise of the authority key has no recovery path. Combined with `admin_sweep`, a compromised authority can still sweep up to `total_deposited - total_withdrawn - rent` before anyone detects it.

[L-04] Deposit-side privacy leak inherent to `DepositReceipt` design (documentation-only)

Severity: LOW (documentation, not a code bug) **File:** `state.rs:194-210` (`DepositReceipt` struct), `ARCHITECTURE.md` "Privacy cost of revoking" section

Description:

Creating a `DepositReceipt` at deposit time establishes an on-chain plaintext record of `{depositor, leaf, amount}`. This record is immediately indexable by any observer and is visible from the moment of deposit, regardless of whether the drop is later claimed, revoked, or the receipt is closed via `close_receipt`. Closing the receipt removes the account from the current chain state but does not erase its contents from historical indexers.

This creates a durable on-chain `depositor ↔ (leaf, amount)` link that did not exist in the legacy 5-account `create_drop` path. In the relayer-mediated legacy flow, the depositor's wallet never signs on-chain; the receipt path requires it to sign, which by itself was a trade-off. The receipt PDA data persists as an additional forensic record beyond the signer's appearance on the create_drop TX.

Scope of leakage (important — this was overstated in the initial M-01 write-up):

- Leaks: depositor wallet, leaf (and therefore amount, and Merkle leaf index).
- Does NOT leak: the nullifier_hash of any subsequent claim, the CreditNote.recipient of any normal claim, or any mapping between this leaf and a future claim_credit TX.

The V2 credit-note circuit continues to hide the leaf↔nullifier_hash mapping. An observer who learns `(depositor, leaf, amount)` from a receipt cannot, through the receipt alone, determine which future `claim_credit` TX corresponds to this leaf. The deposit side is revealed; the claim side remains protected by V2.

Why this is not fixable in code:

Any scheme that hides the receipt from deposit-time observers would require encrypting or commitment-hiding the `{depositor, leaf, amount}` fields. But the depositor pubkey is a Solana account that must exist on-chain (signer of create_drop, recipient of refund on revoke), and the leaf is a public input in the Merkle tree update. Hiding these requires a fundamentally different protocol design (e.g., a ZK-proven "I created some leaf with some amount" without revealing which leaf), which is out of scope.

Recommendation:

- Document the leak clearly in `ARCHITECTURE.md` (done in this audit cycle — see Recommendation 7 in this report).
- Frontend should warn users choosing the receipt path: "your wallet will be publicly linked to this specific drop and amount; use the legacy deposit path if you want maximum claim privacy."

Informational Notes (Audit 04)

[I-01] Note Pool has no revoke mechanism

File: `instructions/deposit_to_note_pool.rs`, `instructions/claim_from_note_pool.rs`

Once a drop's credit note enters the note pool via `deposit_to_note_pool`, the original `DepositReceipt` (if any) no longer corresponds to the pool leaf. The pool leaf is program-constructed from fresh `pool_secret` / `pool_nullifier` / `pool_blinding` provided by the depositor, and there is no receipt-equivalent for the pool layer.

If the user loses `pool_secret` / `pool_nullifier` / `pool_blinding` or the downstream `new_blinding` / `new_salt`, the corresponding treasury obligation is permanently locked. There is no recovery path at the pool layer.

This is a deliberate design trade-off: the note pool's privacy property is precisely the absence of a wallet-to-leaf binding. Adding a receipt-equivalent would reintroduce the linkage that the pool is designed to eliminate.

Recommendation: Document prominently in user-facing materials. Users who prioritize recoverability should not use the note pool. Users who prioritize privacy should use the pool and accept the no-recovery trade-off.

[I-02] Trust-model concern: malicious relayer can set `depositor = relayer` in receipt

File: `instructions/create_drop.rs:66-128`

Description:

The `remaining_accounts` branch of `create_drop` requires `depositor.is_signer`. The stored `receipt.depositor = depositor_info.key()` is whoever signed, not necessarily the actual end-user.

A malicious relayer could offer a "private deposit with revoke" product where the relayer constructs the `create_drop` TX with both `sender = relayer` and `depositor = relayer`. The relayer then holds the revoke option. If the user loses the claim code, only the relayer can reclaim the funds after the 30-day time-lock.

Mitigating factors:

- The on-chain TX is public; the user can verify `receipt.depositor` points to their wallet before trusting the relayer.
- The protocol correctly enforces `depositor.is_signer`, so the relayer cannot forge a depositor claim.
- In the normal relayer flow (documented in `ARCHITECTURE.md`), the user's wallet does not sign on-chain at all; adding a receipt requires it to sign, breaking the privacy-preserving flow.

Recommendation: Frontend and client libraries should refuse to construct relayer-mediated `create_drop` TXs with `depositor != user_wallet`. The direct-deposit flow (user signs as both `sender` and `depositor`) is the intended use of receipts. Document this constraint in the relayer API spec.

[I-03] Carryover: `groth16-solana` 0.0.3 pre-release (Audit 03 I-02)

File: `program/Cargo.lock` (pinned at 0.0.3)

Re-verified: `groth16-solana` is still at 0.0.3. Pinned in `Cargo.lock`, so the version is reproducible. No upgrade has occurred since Audit 02 when this was first noted.

[I-04] Carryover: redundant `fee_recipient` (Audit 03 I-04)

Files: `instructions/claim.rs:160`, `instructions/withdraw_credit.rs:154`

Re-verified: the `fee_recipient` account is still constrained to equal `payer`. The account is therefore a duplicate reference that could be removed to simplify the instruction interface.

Counter Interaction Invariant Analysis (Scope [C])

Invariant under audit: `total_deposited − total_withdrawn = outstanding obligations`, where outstanding is the sum of all amounts committed in live CreditNote PDAs plus pool leaves plus unclaimed DepositReceipts that still correspond to unspent drops. `admin_sweep` refuses to spend if it would break this invariant.

Path-by-path accounting

Path	`total_deposited`	`total_withdrawn`	Notes
`create_drop` (5-account legacy)	**+amount**	—	`create_drop.rs:58-62`
`create_drop` (7-account with receipt)	**+amount**	—	same; receipt creation does not touch counters
`claim` (legacy V1)	—	**+amount**	`claim.rs:78-80`
`claim_credit`	—	—	`claim_credit.rs:78-81` — does not update `total_withdrawn`
`withdraw_credit`	—	**+amount**	`withdraw_credit.rs:104-106`
`deposit_to_note_pool`	—	—	`deposit_to_note_pool.rs` — CreditNote closed; obligation preserved in pool leaf; no counter update
`claim_from_note_pool`	—	—	`claim_from_note_pool.rs:81-84` — fresh CreditNote created; obligation preserved
`revoke_drop`	—	**+amount**	`revoke_drop.rs:75-77`

Each deposit increments `total_deposited` exactly once; each terminal withdrawal (legacy claim, withdraw_credit, revoke_drop) increments `total_withdrawn` exactly once. Multi-hop flows through the note pool preserve the obligation and update `total_withdrawn` only at the final `withdraw_credit`.

Edge cases verified

(a) Drop deposited (no receipt) → claimed via credit note → withdrawn → admin_sweep:

- `total_deposited += amt` at deposit
- `total_withdrawn += amt` at withdraw_credit
- `outstanding = 0` → `admin_sweep` can sweep the full excess → math closes.

(b) Drop deposited with receipt → revoked after 30 days:

- `total_deposited += amt` at deposit
- `total_withdrawn += amt` at revoke_drop
- Receipt PDA closed, rent returned to depositor
- Nullifier PDA created at `[b"nullifier", Poseidon(nullifier)]` — blocks future claims
- `outstanding = 0` → sweep works correctly

(c) Drop deposited with receipt → claimed normally (the M-01 case):

- `total_deposited += amt` at deposit

- `total_withdrawn += amt` at withdraw_credit
- Receipt PDA **orphaned** (M-01)
- Nullifier PDA exists → future `revoke_drop` fails before reaching the receipt close
- `outstanding = 0` → admin_sweep works correctly; counters are balanced
- **Receipt rent (~0.00151 SOL) permanently locked in the orphan PDA.** Not counted in treasury obligation — lives in a separate PDA account.

(d) Note pool round-trip — deposit → claim_credit → deposit_to_note_pool → claim_from_note_pool → withdraw_credit:

- `total_deposited += amt` (once, at base deposit)
- `total_withdrawn += amt` (once, at final withdraw_credit)
- All intermediate steps: no counter change
- Invariant holds throughout; obligation moves through CreditNote → pool leaf → fresh CreditNote → treasury payout

(e) Dishonest leaf over-claim (I-01 design limitation):

- Depositor: `create_drop(amount = 0.01 SOL)` with leaf committing to 100 SOL
- Legitimate others: deposit 99.99 SOL → `total_deposited = 100.00`
- Depositor: claim 100 SOL → `total_withdrawn += 100`
- Treasury balance check permits because treasury has 100 SOL pre-claim
- After: `outstanding = 0.00`, but legitimate users cannot withdraw (treasury drained)
- Counters do not underflow (`total_withdrawn <= total_deposited`)
- `admin_sweep.rs:21-23` uses `checked_sub` — if counters ever did underflow, it would error rather than corrupt state

(f) Concurrent operations:

- Solana serializes writes to the same account within a block. The vault is written by all of: `create_drop`, `claim` (legacy), `withdraw_credit`, `revoke_drop`, `admin_sweep`. Concurrent operations on the vault from different TXs in the same block are serialized by the runtime; later TXs see updated `total_deposited` / `total_withdrawn`. No drift.

admin_sweep math (edge case: receipts present)

`admin_sweep` computes `outstanding = total_deposited - total_withdrawn` and sweeps `treasury - outstanding - rent`. It does **not** account for DepositReceipt rent because that rent is stored in separate PDA accounts, not in the treasury. An admin sweep with outstanding receipts still correctly leaves enough in the treasury to satisfy all outstanding refund obligations (since those refunds are `receipt.amount`, which are already counted in `total_deposited - total_withdrawn`).

Verified: an admin_sweep immediately before a revoke leaves exactly `outstanding + rent` in the treasury. The revoke requires `refund <= outstanding` (enforced in `revoke_drop.rs:72`) AND `refund <= treasury - rent` (enforced at lines 81-85). Both conditions are satisfied because `refund = receipt.amount <= outstanding` and `treasury - rent = outstanding`.

Privacy Analysis of the Revoke Path (Scope [E])

Claim being verified (from `ARCHITECTURE.md` REVOKE INSTRUCTION section): "revoke reveals only the revoking user's own preimage, not other users' data. This publicly links the depositor to a specific unclaimed drop. It does NOT affect the anonymity of OTHER drops in the Merkle tree."

What an observer of a `revoke_drop` TX learns

From the TX data and balance deltas:

- `leaf` — already public in the `DropCreated` event from `create_drop`

- `nullifier_hash = Poseidon(nullifier)` — was private until revoke; now public via the nullifier PDA creation
- `preimage = (secret, nullifier, blinding)` — fully revealed (96-byte opaque field; values extractable from instruction data)
- Signer = depositor pubkey
- Approximate amount — derivable from treasury balance delta AND from the `receipt.amount` field which is plaintext in the receipt PDA (readable until the close instruction in the same TX)

Cross-leaf anonymity impact

Verified claim: No impact on other leaves.

- Each leaf is independently committed. `Poseidon(s, n, amount, blinding)` for one leaf reveals nothing about any other leaf.
- The Merkle tree's root_history contains hashes only, not leaf preimages. A revoke does not leak other leaves' preimages.
- No instruction in the program allows cross-leaf information disclosure.

Clustering analysis (same-depositor linkage)

The **receipt itself** (at deposit time) creates the `depositor → leaf` linkage on-chain. This linkage is:

- Present for all receipt-bearing drops, regardless of whether revoke is ever exercised
- Indexable: anyone can scan for PDAs owned by the program under the `[b"receipt", …]` namespace and read `depositor`, `leaf`, `amount`, `created_at`

A depositor who creates multiple receipts from the same wallet is clusterable across those deposits at deposit time. Revoke adds no new clustering signal; the wallet was already linkable to the leaves via receipts.

This was **not** fully surfaced in the `ARCHITECTURE.md` "Privacy cost of revoking" subsection, which focuses on the revoke-time cost. A more complete statement would be:

> *Creating a DepositReceipt at deposit time establishes a permanent on-chain link between the depositor wallet and a specific (leaf, amount) pair. This linkage exists for the lifetime of the receipt PDA, regardless of whether revoke is ever called. Revoke additionally reveals the leaf preimage. The receipt option trades claim-privacy (via the deposit→leaf linkage) for the fallback capability.*

This is consistent with the ARCHITECTURE.md guidance that "users who prioritize privacy over recoverability should avoid creating receipts" but strengthens it by noting the privacy cost begins at deposit, not at revoke.

Does creating a receipt establish a NEW linkage that didn't exist in the legacy flow?

Yes. In the legacy 5-account `create_drop` call, the only on-chain data tying a wallet to a leaf is the `create_drop` TX itself, where `sender` is the signer. In the relayer-mediated flow, even this linkage is broken: `sender` is the relayer, not the user. The leaf is in the instruction data but the depositor's wallet never appears.

Creating a DepositReceipt adds a separate on-chain record: `receipt.depositor`. This wallet is linkable to the leaf even when the `create_drop` TX itself was relayed. The receipt is therefore a **novel linkage source** relative to the pre-revoke deployment. For receipt-using direct-mode deposits, the information is the same as was already in the TX. For receipt-using relayer-mediated deposits, it is strictly more information.

This is reflected in finding [I-02] above.

New Attack Scenarios Tested (Scope [F])

1. Receipt griefing (non-depositor blocks real depositor's receipt)

Attack: Attacker front-runs a legitimate depositor's `create_drop` and allocates a `DepositReceipt` for the victim's intended leaf. Victim cannot create their receipt and loses the revoke option.

Result: NOT EXPLOITABLE. Receipt PDA seed is `[b"receipt", leaf]` where `leaf = Poseidon(secret, nullifier, amount, blinding)`. The attacker must know the leaf preimage to compute the same PDA. The preimage is the depositor's secret; the attacker cannot guess it (Poseidon preimage resistance). A different-preimage leaf produces a different PDA; no collision.

2. Receipt squatting after normal claim

Attack: After a drop is claimed normally, the orphan DepositReceipt persists. An attacker tries to exploit the leftover PDA for some purpose.

Result: NOT DIRECTLY EXPLOITABLE FOR FUND THEFT. The orphan cannot be used by the attacker (it's owned by the program; Anchor's `close = depositor` requires the depositor's signature). However, it is a **privacy regression** (see M-01): the receipt's plaintext fields remain readable forever, providing durable depositor→leaf→recipient linkage.

3. Feature flag leakage (short-revoke-timeout in production binary)

Attack: A malicious build includes the `short-revoke-timeout` feature in the production binary, shipping a 5-second timeout instead of 30 days.

Result: NOT PRESENT in the currently-built production binary.

Verification methodology:

- Built with `cargo build-sbf` (no feature flag) → binary contains bytes `00 8d 27 00` (little-endian u32 encoding of 2,592,000)
- Built with `cargo build-sbf --features short-revoke-timeout` → binary does NOT contain those bytes
- Bytewise diff between the two binaries: 25,508 bytes differ, consistent with const propagation; sizes are identical (601,576 bytes) because the only change is a 64-bit immediate
- `strings` on the production binary shows no "short-revoke-timeout" or related test-specific identifiers

Operational recommendation: CI should build the release binary with `cargo build-sbf` (no features) and reject any binary whose hex dump contains a suspicious small integer in place of `0x0000000000278D00`. The current deploy process in `ARCHITECTURE.md` documents the correct (featureless) build command.

4. Preimage substitution (valid preimage → different nullifier_hash)

Attack: Depositor submits a `preimage` that reconstructs to the correct `leaf` using `receipt.amount`, but produces a `nullifier_hash` different from what a legitimate claimer would use — thereby writing to a different nullifier PDA and leaving the claim path open.

Result: NOT EXPLOITABLE. Poseidon is collision-resistant. The constraints are:

- `Poseidon(s, n, receipt.amount, b) == receipt.leaf` — since `receipt.leaf` is fixed and `receipt.amount` is read from on-chain state (not user-supplied), and Poseidon is collision-resistant, the valid (s, n, b) tuple is unique.
- `nullifier_hash = Poseidon(n)` — since `n` is uniquely determined by the previous constraint, `nullifier_hash` is also uniquely determined.

The depositor cannot produce a valid preimage that both satisfies the leaf constraint and yields a different nullifier_hash without breaking Poseidon's collision resistance. Note: even if they tried to submit a `nullifier_hash` arg that doesn't match `Poseidon(n)`, the explicit check `computed_null_hash == nullifier_hash` at `revoke_drop.rs:68-72` fails (CommitmentMismatch).

5. Cross-PDA seed collision (leaf bytes producing receipt/nullifier/credit/pool_nullifier collision)

Attack: Craft a `leaf` value such that `[b"receipt", leaf]` derives the same PDA as `[b"nullifier", Y]` for some Y, enabling one to masquerade as the other.

Result: NOT EXPLOITABLE. The PDA prefixes are byte-different:

- `b"receipt"` (7 bytes)
- `b"nullifier"` (9 bytes)
- `b"credit"` (6 bytes)
- `b"pool_nullifier"` (14 bytes)
- `b"note_pool_tree"` (14 bytes)

Solana PDA derivation hashes the entire seed array (each seed as a distinct byte string) plus the program ID. Two different seed arrays cannot produce the same PDA without a SHA-256 collision on the derivation input. Additionally, the 32-byte component in each seed (leaf, nullifier_hash, pool_nullifier_hash) is either a Poseidon output or a free-choice leaf; cross-namespace collisions would require a preimage attack against Poseidon AND/OR a second preimage attack against SHA-256 in PDA derivation.

6. V3 public-input order manipulation

Attack: Rearrange or permute the 4 V3 public inputs to bypass proof verification.

Result: NOT EXPLOITABLE. `claim_from_note_pool.rs:51-56` builds the public inputs array in the exact order the V3 circuit declares signals: `pool_merkle_root, pool_nullifier_hash, new_stored_commitment, recipient_hash`. `vk.rs:213` declares `V3_IC: [[u8; 64]; 5]` (4 inputs + 1 constant) and `V3_NR_PUBLIC_INPUTS: usize = 4`, matching the Groth16 verifier's expectation. No IC-array size mismatch.

7. Pool layer dishonest leaf insertion

Attack: A user deposits to the note pool with a leaf whose committed amount exceeds the verified credit note amount.

Result: NOT EXPLOITABLE at the pool layer. `deposit_to_note_pool.rs:55-63` opens the credit note commitment on-chain and verifies `Poseidon(Poseidon(amount, blinding), salt) == credit.commitment`. The program then constructs the pool leaf at line 69 as `poseidon_hash_4(&pool_secret, &pool_nullifier, &amount_bytes, &pool_blinding)` using the **verified** amount. A user cannot insert a pool leaf whose committed amount differs from the verified credit note amount.

Caveat: if the credit note itself was created from a dishonest base-layer leaf (Audit 02's H-02, downgraded to I-01 design limitation), the inflated amount propagates through to the pool. This is a base-layer property, not a pool-layer weakness.

8. Cross-namespace nullifier replay (pool_nullifier as base nullifier)

Attack: Use a pool_nullifier_hash to derive a PDA in the base `[b"nullifier", ...]` namespace, consuming one to block the other.

Result: NOT EXPLOITABLE. Different prefixes (`b"pool_nullifier"` vs `b"nullifier"`) produce different PDAs. An attacker cannot cause base-nullifier consumption via pool actions or vice versa.

9. Revoke before time-lock expires

Attack: Call `revoke_drop` immediately after `create_drop`.

Result: BLOCKED by `RevokeTooEarly` (6014). `revoke_drop.rs:49-52` checks `Clock::get()?.unix_timestamp >= receipt.created_at + REVOKE_TIMEOUT`. Production binary uses `REVOKE_TIMEOUT = 2,592,000` (30 days).

10. Revoke by non-depositor

Attack: A non-depositor signs and tries to revoke.

Result: BLOCKED by `InvalidDepositReceipt` / `UnauthorizedRevoke`. `revoke_drop.rs:41-46` uses `require_keys_eq!(depositor.key(), receipt.depositor, UnauthorizedRevoke)`. Separately, `create_drop.rs:74-77` uses `InvalidDepositReceipt` for signer and PDA-mismatch checks during receipt creation.

11. Double-revoke (same drop revoked twice)

Attack: Call `revoke_drop` twice for the same leaf.

Result: BLOCKED. First revoke closes the receipt (`close = depositor`), so the second call fails on the receipt PDA lookup (`AccountNotInitialized`). Separately, the first revoke creates the nullifier PDA, so even without the receipt lookup the `init` on nullifier_account would fail.

12. Revoke after claim

Attack: Legitimate drop is claimed; depositor then tries to also revoke.

Result: BLOCKED. The nullifier PDA was created at claim time; `revoke_drop`'s `init` constraint on `nullifier_account` fails ("already in use"). The M-01 orphan receipt remains but cannot be used to drain funds.

13. Cross-receipt preimage attack

Attack: Depositor has two active receipts A and B with different amounts. They submit a `revoke_drop` targeting receipt A's PDA but provide preimage from receipt B (hoping the stored amount_A will somehow combine with preimage_B to verify).

Result: BLOCKED by `CommitmentMismatch` . The program reads `amount` from `receipt_A.amount` and reconstructs `Poseidon(secret_B, nullifier_B, amount_A, blinding_B)` . This does not equal `receipt_A.leaf = Poseidon(secret_A, nullifier_A, amount_A, blinding_A)` . Verified by `scripts/security-revoke-tests.js` test F.

14. Receipt reallocation via close+recreate

Attack: Depositor revokes successfully (closing the receipt), then tries to `create_drop` again with the same leaf to create a fresh receipt.

Result: BLOCKED. `create_drop` with the same leaf would succeed in creating a new receipt PDA (the old one was closed), BUT the new drop's leaf is now a duplicate in the Merkle tree. The tree itself allows duplicates (no uniqueness check on leaves), so the second deposit succeeds. However, the revoke-path nullifier PDA from the first revoke blocks any future `revoke_drop` or `claim_credit` against this leaf (shared nullifier namespace, same preimage → same nullifier_hash → nullifier PDA exists). So: the second deposit is effectively a loss — the depositor deposits SOL that can never be withdrawn (claim blocked by existing nullifier; revoke blocked similarly).

This is a self-inflicted loss, not an attack vector. But it is a **footgun** worth documenting: "Do not reuse the same leaf preimage across deposits."

Recommendation: Frontend enforces fresh `(secret, nullifier, blinding)` per drop. This is the default behavior of client libraries and is unlikely to occur accidentally.

Instruction-by-Instruction Review (new and modified since Audit 03)

`initialize_note_pool`

Check	Result
Authority is Signer	□
Vault `has_one = authority`	□
`NotePool` PDA `init`	□ `seeds = [b"note_pool"]`
`NotePoolTree` PDA `init`	□ `seeds = [b"note_pool_tree", vault.key()]`
`ZERO_HASHES` in `filled_subtrees`	□ all 20 levels initialized

`root_history` full initialization	□ Only slot 0 [**L-01**]
Cannot be called twice	□ `init` on both accounts

`deposit_to_note_pool`

Check	Result
Opening length validated	□ 72 bytes (amount + blinding + salt)
Pool params length validated	□ 96 bytes
Credit note recipient matches signer	□ `recipient: Signer<'info>` AND `key() == credit.recipient`
Amount > 0	□
Re-randomized commitment verified	□ `Poseidon(Poseidon(amt, b), salt) == credit.commitment`
Pool leaf constructed with VERIFIED amount	□ `poseidon_hash_4(pool_secret, pool_nullifier, amount_bytes, pool_blinding)` at line 69 — **eliminates dishonest leaf at pool layer**
Pool tree append respects capacity	□ `next_index < 2^20` in `note_pool_tree_append`
CreditNote closed on success	□ `close = payer`
`note_pool.total_deposits` overflow	□ `checked_add(1)`

`claim_from_note_pool`

Check	Result
Input length validated	□ 64 bytes (pool_merkle_root + new_stored_commitment)
Pool root in history	□ `is_known_root`
Groth16 V3 proof verified	□ 4 public inputs, verified against `verifying_key_v3()`
Recipient bound by proof	□ `Poseidon(pubkey_hi, pubkey_lo)` as public input [3]
Pool nullifier PDA `init`	□ `seeds = [b"pool_nullifier", pool_nullifier_hash]` — disjoint from base nullifier namespace
Fresh CreditNote `init`	□ `seeds = [b"credit", pool_nullifier_hash]` — shares CreditNote namespace with `claim_credit` but collision-resistant
Zero SOL moves	□
`note_pool.total_claims` overflow	□ `checked_add(1)`
`credit.salt` field populated	□ Pseudorandom derivation from `pool_nullifier_hash + new_stored_commitment` for namespace indistinguishability (not used for verification)

`create_drop` (extended with `remaining_accounts`)

Check	Result

Amount validation (min + cap)	□ unchanged from pre-revoke
CPI transfer	□ unchanged
Legacy 5-account path	□ Verified end-to-end via `scripts/legacy-create-drop-test.js`
Receipt branch: depositor is signer	□ `InvalidDepositReceipt`
Receipt branch: depositor and receipt writable	□ `InvalidDepositReceipt`
Receipt branch: PDA derivation	□ `Pubkey::find_program_address(&[b"receipt", leaf], program_id)` + `require_keys_eq!`
Receipt branch: duplicate-leaf prevention	□ `lamports() == 0 && data_is_empty()` → `LeafAlreadyDeposited`
Receipt branch: `system_program::create_account` signed by PDA seeds	□
Receipt branch: discriminator + Borsh serialization	□ Anchor-compatible
`total_deposited` overflow	□ `checked_add`

`close_receipt` (added during this audit cycle as the M-01 fix)

Check	Result
Depositor is Signer	□ `depositor: Signer<'info>`
Receipt PDA derivation	□ `seeds = [b"receipt", leaf.as_ref()], bump = deposit_receipt.bump`
Explicit `depositor == receipt.depositor` check	□ `require_keys_eq!(... InvalidDepositReceipt)` — essential because `close = depositor` alone does not verify this
Close returns rent to depositor	□ `close = depositor`
`DepositReceiptClosed` event	□ emits `leaf`, `depositor`, `timestamp`
No nullifier-state check	□ intentional; depositor judges off-chain when to close
Idempotent / double-close	□ Second close fails `AccountNotInitialized` (receipt PDA gone after first close)

`revoke_drop`

Check	Result
Preimage length validated	□ 96 bytes
Receipt PDA derivation	□ `seeds = [b"receipt", leaf], bump = deposit_receipt.bump`
Receipt `leaf` field matches arg	□ `require!(receipt.leaf == leaf, UnauthorizedRevoke)` — belt-and-suspenders on the seed derivation
Depositor is signer	□
Depositor key matches `receipt.depositor`	□ `require_keys_eq!(...UnauthorizedRevoke)`
Time-lock enforced	□ `now >= receipt.created_at + REVOKE_TIMEOUT`, `checked_add` on the addition

Leaf preimage reconstructed on-chain	`poseidon_hash_4(secret, nullifier, u64_to_field_be(receipt.amount), blinding) == receipt.leaf`
Nullifier hash reconstructed on-chain	`poseidon_hash_1(nullifier) == nullifier_hash` arg — prevents nullifier_hash substitution
Nullifier PDA `init` (shared namespace)	`seeds = [b"nullifier", nullifier_hash]` — mutex with `claim_credit`
Obligation-aware refund bound	`refund <= outstanding` (line 72) AND `refund <= treasury - rent_exempt_min` (line 85)
Direct lamport transfer	no CPI
`total_withdrawn` overflow	`checked_add`
Receipt closed on success	`close = depositor` — rent returned
`DropRevoked` event	emits `leaf`, `depositor`, `timestamp` (no amount)
`REVOKE_TIMEOUT` feature-gated	`#[cfg(feature = "short-revoke-timeout")]` at `state.rs:19-23`

Summary Table

ID	Severity	Title	Status
M-01 (this audit)	MEDIUM	Orphan `DepositReceipt` after normal claim — rent lock	**FIXED** in this cycle via `close_receipt` instruction
L-01 (this audit)	LOW	`note_pool_tree` root history zero-initialized	**Open**
L-02 (carryover from Audit 03 L-01-NEW)	LOW	No `drop_cap` validation	**Open**
L-03 (carryover from Audit 03 L-02-NEW)	LOW	No authority rotation	**Open**
L-04 (this audit)	LOW	Deposit-side privacy leak inherent to receipt design	**Documentation-only; cannot be fixed in code**
I-01 (this audit)	INFO	Note pool has no revoke mechanism	**Accepted design**
I-02 (this audit)	INFO	Malicious relayer can set depositor = relayer	**Trust model**
I-03 (carryover from Audit 03 I-02)	INFO	`groth16-solana` 0.0.3 pre-release	**Pinned in Cargo.lock**
I-04 (carryover from Audit 03 I-04)	INFO	Redundant `fee_recipient`	**Open**

Fixes re-verified in this audit:

ID	Severity	Title	Current status
H-01-NEW (Audit 03)	~~HIGH~~	`admin_sweep` obligation tracking	**FIXED** (re-verified in this audit)

M-01-NEW (Audit 03)	~~MEDIUM~~	CreditNote commitment re-randomization	✅ **FIXED** (re-verified)
M-02-NEW (Audit 03)	~~MEDIUM~~	`admin_sweep` event	✅ **FIXED** (re-verified)
M-03-NEW (Audit 03)	~~MEDIUM~~	`DropCreated` event leakage	✅ **FIXED** (re-verified)

Recommendations Priority

Before Mainnet (Blocking)

1. ~~**[M-01]** Ship `close_receipt` instruction.~~ ✅ **FIXED in this audit cycle.** New `close_receipt` instruction merged; see M-01 Fix subsection. E2E + 5 security tests pass on localnet.
2. Update `audits/README.md` fix tracker to mark H-01-NEW, M-01-NEW, M-02-NEW, M-03-NEW as FIXED (they are currently still listed as "Open" in the tracker, even though Audit 03 already verified them fixed). Done in this audit cycle.

Short-Term

3. **[L-01]** Initialize all 30 slots of `note_pool_tree.root_history` to `ZERO_HASHES[MERKLE_DEPTH]` in `initialize_note_pool`. Combine with the existing L-03-NEW fix for the main Merkle tree.
4. **[L-02, L-03]** Close out carryover LOWs from Audit 02/03: add `drop_cap` validation; add `update_authority` propose/accept pattern.

Medium-Term

5. **[I-02]** Frontend / client-library guardrail: refuse to construct relayer-mediated `create_drop` TXs with `depositor != connected_wallet`. Document in relayer API spec.
6. **[I-04]** Remove redundant `fee_recipient` account from `claim` and `withdraw_credit` since the constraint makes it equal to `payer`. Saves one account per TX.

Documentation

7. **[L-04]** Extend the ARCHITECTURE.md "Privacy cost of revoking" subsection to state clearly that the deposit-side linkage is created at **deposit** (when the receipt is created), not only at revoke. The prior wording implied the cost was revoke-specific; in fact, creating the receipt at all is a privacy-reducing action, and `close_receipt` does not undo it (indexers retain history). **Done in this audit cycle** — see the updated wording in the "Privacy cost of revoking" section.
8. Document the I-01 pool layer recovery limitation prominently.
9. Document the I-02 relayer trust-model constraint.

Conclusion

The V3 Note Pool and `revoke_drop` features are **soundly implemented**. The Option C preimage-verification design for revoke correctly pins the depositor to the exact leaf preimage, closing the double-payout window that would have existed under the earlier Option A design. The note pool's on-chain opening verification correctly eliminates dishonest leaves at the pool layer (a strict improvement over the base layer's I-01 limitation for deposits that flow through the pool).

The core security properties — Groth16 ZK verification, nullifier mutex across claim/revoke, treasury obligation accounting, direct-lamport privacy on refunds — are preserved. The `short-revoke-timeout` Cargo feature is correctly gated and does not leak into the production binary.

The single MEDIUM finding (M-01) was an orphaned-receipt rent lock. It was identified, rigorously analyzed (including a correction to an initial privacy-chain overstatement), and **fixed during this audit cycle** with the new `close_receipt` instruction. The fix was chosen after carefully rejecting alternatives that would have introduced new deposit↔claim linkage (re-seeding by `nullifier_hash`, dual PDAs keyed by nullifier, or touching the receipt inside `claim_credit`). All test coverage (E2E + five security-adversarial tests G–K) passes.

The remaining privacy cost of the receipt mechanism — a deposit-time `depositor ↔ (leaf, amount)` linkage visible to indexers — is inherent to the design and cannot be removed in code. It is now classified as a separate documentation-only LOW finding (L-04) and the ARCHITECTURE.md "Privacy cost of revoking" section has been strengthened to state this explicitly so users can make an informed choice between the receipt path (recoverability + deposit-side linkage) and the legacy 5-account path (privacy, no recovery option).

No CRITICAL findings. No HIGH findings. No actively-exploitable vulnerabilities. The single MEDIUM was fixed within this cycle.

End of Audit 04 report.